

Support Atelier

ASI

Chapitre 2 : Spring Data JPA - Le mapping des associations

SOMMAIRE

I.	Une relation du véhicule	2
II.	Encore sur la véhicule.....	2
III.	Toutes les relations.....	3

I. Une relation du véhicule

1. Ajouter les attributs nécessaires pour établir la relation **OnToMany/ManyToOne** entre véhicule et agence.
2. Déclarer la relation en utilisant **mappedBy**. Sachant que :
 - La véhicule peut exister sans agence
 - L'ajout d'une véhicule implique la présence des ces agences
 - Le chargement d'une agence implique le chargement des véhicules
3. Lancer l'application et vérifier qu'il y a une colonne qui a été ajoutée.
4. Au niveau des tests, ajouter la classe suivante :

```
@RunWith(SpringJUnit4ClassRunner.class)
@SpringBootTest
public class AgenceTests {
    @Autowired
    private AgenceRepositoryMock agenceRepository;
}
```

5. Dans le même fichier, ajouter l'interface **AgenceRepositoryMock** qui hérite de **CrudRepository**.
6. Dans cette même classe, ajouter une méthode nommée **addAgence** et la marquer comme méthode de test via l'annotation **@Test**.
7. Au niveau de cette méthode créer l'agence suivante :

123 id_agence	A-Z adresse	A-Z nom	A-Z telephone	A-Z ville
8	1 Rue Hedi	Agence ariana	71585874	Tunis

qui a les deux véhicules suivantes

123 id_vehicule	A-Z categorie	A-Z immatriculation	A-Z marq	A-Z mod	A-Z statut	123 tarif_journalier
12	SUV	785414TU96	Isuzu	DMax	MAINTENANCE	100
13	UTILITAIRE	785414TU95	Toyota	Yaris	DISPONIBLE	80

8. En utilisant la méthode **save** de **CrudRepository**, persister les données.
9. Exécuter le test et vérifier que les données ont été bien insérées dans la base de données.
10. Ré-exécuter à nouveau le test et vérifier que le test échoue.

II. Encore sur la véhicule

11. Au niveau de la classe de tests, ajouter une méthode nommée **loadAgence**.
12. Dans cette méthode récupérer la liste des toutes les agences via la méthode **findAll**.
13. Parcourir le résultat et en utilisant la classe **StringBuilder**, ajouter les informations sur :
 - l'identifiant de l'agence
 - Le nom de l'agence

- Le nombre de véhicules attaché à chaque véhicule
 - Identifiant et immatriculation de chaque véhicule
14. En utilisant une assertion qui échoue toujours, affiche le résultat.
15. Vérifier que le résultat ressemble à ceci :

```
org.opentest4j.AssertionFailedError:
8 | Agence ariana
Vehicules Count : 2
=== 13|785414TU95
=== 12|785414TU96
```

III. Toutes les relations

16. Identifier et implémenter toutes les relations **OnToMany/ManyToOne** bidirectionnelles, sachant que :
- Le chargement d'une véhicule n'implique pas le chargement des réservations
 - La suppression d'une véhicule implique la suppression de ses réservations
 - Le chargement d'une agence n'implique pas le chargement des employés
 - La suppression d'une agence n'implique pas la suppression des employés associés
 - Le chargement d'un client implique le chargement de ces réservations
 - La suppression d'un client implique la suppression de ces réservations
17. Exécuter et vérifier la création des colonnes nécessaires.
18. Implémenter la relation entre contrat et paiement, sachant que le chargement du contrat implique le chargement des paiements. Tester la création.
19. Implémenter la relation entre maintenance et véhicule puis tester la création de la colonne.
20. Implémenter la relation entre réservation et contrat en mettant la clé dans la table associée à la réservation. Tester la création.
21. Implémenter la relation entre équipement et véhicule sachant que les chargements et les suppressions ne sont pas liées. Tester la création de la table de jointure.
22. Au niveau du dossier **autoloc-api**, copier le contenu de **.gitignore** dans le fichier **.gitignore** du racine du projet. Supprimer le fichier **.gitignore** du sous dossier. Valider les changements avec le message « Chap3 » et pousser sur **GitHub**.